

✓ Theoretical Questions



Question 1: Mathematical Principles of Clustering (K-Means)

Given a dataset $X = \{x_1, \dots, x_N\}$, the K-Means algorithm partitions the data into K clusters by minimizing the **Within-Cluster Sum of Squares (WCSS)** objective function:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2$$

where:

- μ_k is the centroid of cluster k ,
- $r_{nk} \in \{0, 1\}$ is an indicator variable,
- $\sum_{k=1}^K r_{nk} = 1$ for each n .

1. K-Means as Coordinate Descent

The objective J depends on two blocks of variables:

- Assignments $R = \{r_{nk}\}$,
- Centroids $M = \{\mu_k\}$.

The standard K-Means algorithm alternates between minimizing J with respect to one block while keeping the other fixed. This is **block coordinate descent**.

1 (a): E-Step (Assignment)

Fix the centroids μ_k . Then the objective becomes:

$$J = \sum_{n=1}^N \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2.$$

For a fixed data point x_n , we minimize:

$$J_n = \sum_{k=1}^K r_{nk} \|x_n - \mu_k\|^2$$

subject to:

- $r_{nk} \in \{0, 1\}$,
- $\sum_{k=1}^K r_{nk} = 1$.

Since exactly one r_{nk} must be equal to 1, the minimum is obtained by choosing the closest centroid:

$$k^*(n) = \arg \min_{k \in \{1, \dots, K\}} \|x_n - \mu_k\|^2,$$
$$r_{nk} = \begin{cases} 1, & k = k^*(n), \\ 0, & \text{otherwise.} \end{cases}$$

Thus, the E-step minimizes J with respect to R while keeping μ_k fixed.

1 (b): M-Step (Centroid Update Step)

Fix the assignments r_{nk} . Then J decomposes over clusters:

$$J = \sum_{k=1}^K J_k(\mu_k), \quad J_k(\mu_k) = \sum_{n=1}^N r_{nk} \|x_n - \mu_k\|^2.$$

Each centroid μ_k can be optimized independently.

2. Proof that the Arithmetic Mean Minimizes the M-Step

Consider one cluster k :

$$J_k(\mu_k) = \sum_{n=1}^N r_{nk} \|x_n - \mu_k\|^2.$$

Expand the squared norm:

$$\|x_n - \mu_k\|^2 = (x_n - \mu_k)^\top (x_n - \mu_k) = x_n^\top x_n - 2x_n^\top \mu_k + \mu_k^\top \mu_k.$$

Substitute into J_k :

$$J_k(\mu_k) = \sum_{n=1}^N r_{nk} x_n^\top x_n - 2 \sum_{n=1}^N r_{nk} x_n^\top \mu_k + \sum_{n=1}^N r_{nk} \mu_k^\top \mu_k.$$

Let $N_k = \sum_{n=1}^N r_{nk}$ be the number of points assigned to cluster k . Then:

$$J_k(\mu_k) = \text{const} - 2 \left(\sum_{n=1}^N r_{nk} x_n \right)^\top \mu_k + N_k \mu_k^\top \mu_k.$$

Taking the Derivative

Using:

- $\nabla_\mu(\mu^\top \mu) = 2\mu$,
- $\nabla_\mu(a^\top \mu) = a$,

we obtain:

$$\nabla_{\mu_k} J_k(\mu_k) = -2 \sum_{n=1}^N r_{nk} x_n + 2N_k \mu_k.$$

Setting the derivative equal to zero:

$$\begin{aligned} -2 \sum_{n=1}^N r_{nk} x_n + 2N_k \mu_k &= 0 \\ \Rightarrow N_k \mu_k &= \sum_{n=1}^N r_{nk} x_n \\ \Rightarrow \mu_k &= \frac{\sum_{n=1}^N r_{nk} x_n}{\sum_{n=1}^N r_{nk}} \end{aligned}$$

This is exactly the **arithmetic mean of the points assigned to cluster k** .

Conclusion

- The E-step and M-step alternately minimize J with respect to R and μ .
- Each step does not increase the objective value.
- Therefore, standard K-Means is a **block coordinate descent algorithm** that converges to a local minimum of the WCSS objective.

Question 2: SVM and the Kernel Trick

We have four points in $\mathbb{R}^2$:

Class (-1)	Class (+1)
(0, 0)	(1, 0)
(1, 1)	(0, 1)

So the two (-1) points lie on one diagonal of the unit square, and the two $(+1)$ points lie on the other diagonal.

This is the classic **XOR** pattern.

```
import numpy as np
import matplotlib.pyplot as plt

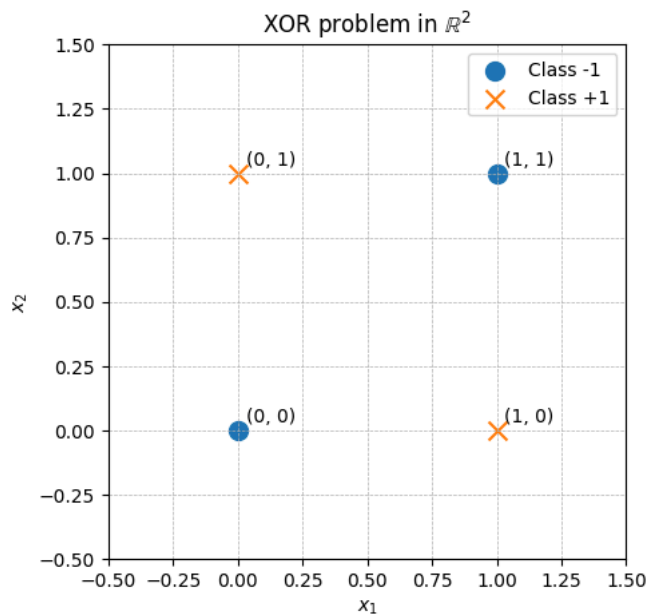
# XOR data
X_neg = np.array([[0,0],[1,1]]) # Class -1
X_pos = np.array([[1,0],[0,1]]) # Class +1

plt.figure(figsize=(5,5))
plt.scatter(X_neg[:,0], X_neg[:,1], s=100, marker='o', label='Class -1')
plt.scatter(X_pos[:,0], X_pos[:,1], s=100, marker='x', label='Class +1')

for p in X_neg:
    plt.text(p[0]+0.03, p[1]+0.03, f"({int(p[0]}), {int(p[1])})")
for p in X_pos:
    plt.text(p[0]+0.03, p[1]+0.03, f"({int(p[0]}), {int(p[1])})")

plt.xlim(-0.5, 1.5)
```

```
plt.ylim(-0.5, 1.5)
plt.grid(True, linestyle='--', linewidth=0.5)
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.title("XOR problem in $\mathbb{R}^2$")
plt.legend()
plt.show()
```



1) Linear Inseparability (Geometric Explanation)

A linear hard-margin SVM in $\mathbb{R}^2$ tries to find a line

$$w^T x + b = 0$$

A linear hard-margin SVM searches for a straight line that places all points of one class on one side and all points of the other class on the opposite side.

Geometrically this is impossible because:

- each class occupies **opposite corners** of the square resulting in **convex hull** overlap. If convex hulls overlap, no linear separator exists,
- any line that puts (0, 0) and (1, 1) on the same side will necessarily put **one** of (1, 0) or (0, 1) on that same side as well (and vice versa).

Therefore, a hard-margin linear SVM cannot separate the classes without error.

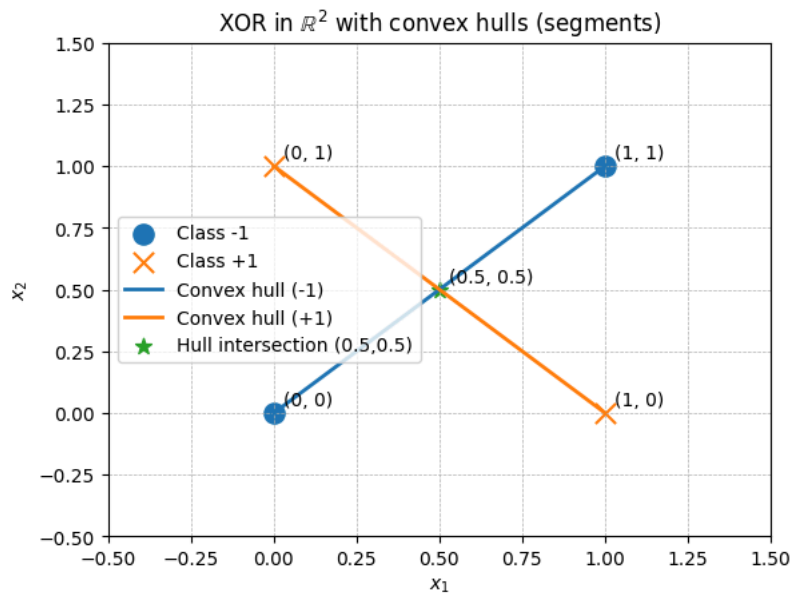
```
# Points
plt.scatter(X_neg[:,0], X_neg[:,1], s=120, marker='o', label='Class -1')
plt.scatter(X_pos[:,0], X_pos[:,1], s=120, marker='x', label='Class +1')

# Convex hulls
plt.plot(X_neg[:,0], X_neg[:,1], linewidth=2, label='Convex hull (-1)')
plt.plot(X_pos[:,0], X_pos[:,1], linewidth=2, label='Convex hull (+1)')

# Mark intersection of the two hull segments
plt.scatter([0.5], [0.5], s=80, marker='*', label='Hull intersection (0.5,0.5)')
plt.text(0.5+0.03, 0.5+0.03, "(0.5, 0.5)")

for p in np.vstack([X_neg, X_pos]):
    plt.text(p[0]+0.03, p[1]+0.03, f"({int(p[0])}, {int(p[1])})")

# Styling
plt.xlim(-0.5, 1.5)
plt.ylim(-0.5, 1.5)
plt.grid(True, linestyle='--', linewidth=0.5)
plt.xlabel("$x_1$")
plt.ylabel("$x_2$")
plt.title("XOR in $\mathbb{R}^2$ with convex hulls (segments)")
plt.legend()
plt.show()
```



2. Feature Mapping into $\mathbb{R}^3$

Given

$$\phi(x_1, x_2) = (x_1, x_2, x_1 x_2).$$

Map each point in $\mathbb{R}^3$:

Original x	Label y	$\phi(x)$
(0,0)	-1	(0,0,0)
(1,1)	-1	(1,1,1)
(1,0)	+1	(1,0,0)
(0,1)	+1	(0,1,0)

```
# Original XOR points in R^2
X = np.array([[0,0],[1,1],[1,0],[0,1]])
y = np.array([-1,-1, 1, 1]) # labels: -1 for (0,0),(1,1); +1 for (1,0),(0,1)

# Feature mapping in R^3
phi = np.c_[X, X[:,0]*X[:,1]]

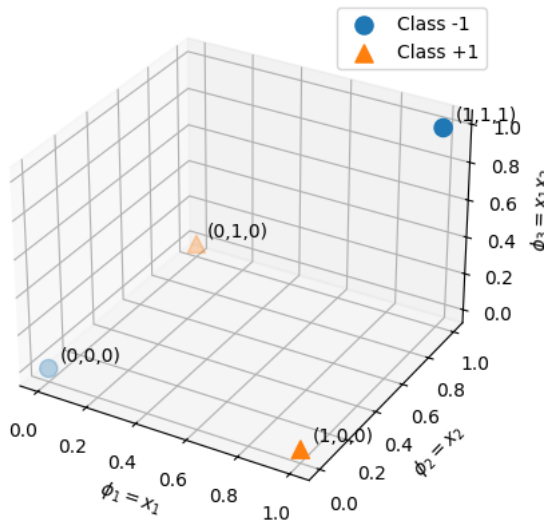
# 3D scatter plot of mapped points
fig = plt.figure(figsize=(7,5))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(*phi[y==-1].T, s=90, marker='o', label='Class -1')
ax.scatter(*phi[y== 1].T, s=90, marker='^', label='Class +1')

for z in phi:
    ax.text(z[0]+0.03, z[1]+0.03, z[2]+0.03, f"({int(z[0])},{int(z[1])},{int(z[2])})")

ax.set_xlabel("$\\phi_1 = x_1$")
ax.set_ylabel("$\\phi_2 = x_2$")
ax.set_zlabel("$\\phi_3 = x_1x_2$")
ax.set_title("XOR points mapped to $\\mathbb{R}^3$ via $\\phi(x_1,x_2)=(x_1,x_2,x_1x_2)$")
ax.legend()
plt.show()
```

XOR points mapped to $\mathbb{R}^3$ via $\phi(x_1, x_2) = (x_1, x_2, x_1x_2)$



3. Linear Separability in $\mathbb{R}^3$ High Demension

We want a hyperplane

$$w^T \phi(x) + b = 0$$

that classifies correctly, i.e.

$$y(w^T \phi(x) + b) > 0$$

for all four points.

```
X = np.array([[0,0],[1,1],[1,0],[0,1]])
y = np.array([-1,-1,1,1])
Phi = np.c_[X, X[:,0]*X[:,1]]

w0, b0 = np.array([1,1,-2]), -0.5

for i in range(1, 5):
    w, b = i*w0, i*b0
    print(f"{i}) w={w}, b={b}")
```

```
1) w=[ 1  1 -2], b=-0.5
2) w=[ 2  2 -4], b=-1.0
3) w=[ 3  3 -6], b=-1.5
4) w=[ 4  4 -8], b=-2.0
```

A working choice is:

$$w = \begin{pmatrix} 1 \\ 1 \\ -2 \end{pmatrix}, \quad b = -\frac{1}{2}.$$

Let $f(x) = w^T \phi(x) + b$.

$$1. \ x = (0,0), y = -1$$

$$\phi(x) = (0,0,0)$$

$$f(x) = 1 \cdot 0 + 1 \cdot 0 - 2 \cdot 0 - \frac{1}{2} = -\frac{1}{2} < 0$$

Correct for class (-1) .

$$2. \ x = (1,1), y = -1$$

$$\phi(x) = (1,1,1)$$

$$f(x) = 1 \cdot 1 + 1 \cdot 1 - 2 \cdot 1 - \frac{1}{2} = 2 - 2 - \frac{1}{2} = -\frac{1}{2} < 0$$

Correct for class (-1) .

$$3. \ x = (1,0), y = +1$$

$$\phi(x) = (1,0,0)$$

$$f(x) = 1 \cdot 1 + 1 \cdot 0 - 2 \cdot 0 - \frac{1}{2} = \frac{1}{2} > 0$$

Correct for class $(+1)$.

$$4. \ x = (0,1), y = +1$$

$$\phi(x) = (0,1,0)$$

$$f(x) = 1 \cdot 0 + 1 \cdot 1 - 2 \cdot 0 - \frac{1}{2} = \frac{1}{2} > 0$$

Correct for class (+1).

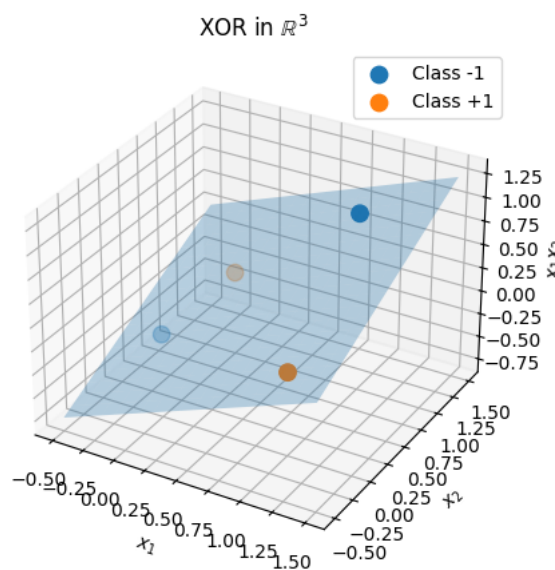
```
X = np.array([[0,0],[1,1],[1,0],[0,1]])
y = np.array([-1,-1,1,1])
phi = np.c_[X, X[:,0]*X[:,1]]

fig = plt.figure(figsize=(6,5))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(*phi[y==-1].T, s=80, label='Class -1')
ax.scatter(*phi[y== 1].T, s=80, label='Class +1')

w, b = np.array([1,1,-2]), -0.5
u, v = np.meshgrid(np.linspace(-.5,1.5,20),
                    np.linspace(-.5,1.5,20))
_ = ax.plot_surface(u, v, -(w[0]*u + w[1]*v + b)/w[2], alpha=.3)

ax.set(xlabel='$x_1$', ylabel='$x_2$', zlabel='$x_1x_2$',
       title='XOR in $\mathbb{R}^3$')
ax.legend()
plt.show()
```



Question 3: PCA Manual Training

We are given a centered dataset with 3 features. The empirical covariance matrix Σ is calculated as follows:

$$\Sigma = \begin{pmatrix} 3 & 2 & 0 \\ 2 & 3 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

1. Characteristic Equation

We compute the characteristic equation: $\det(\Sigma - \lambda I) = 0$

$$\Sigma - \lambda I = \begin{pmatrix} 3-\lambda & 2 & 0 \\ 2 & 3-\lambda & 0 \\ 0 & 0 & 2-\lambda \end{pmatrix}$$

Because the matrix is block-diagonal (we have zeros), the determinant factorizes:

$$\det(\Sigma - \lambda I) = \det \begin{pmatrix} 3-\lambda & 2 \\ 2 & 3-\lambda \end{pmatrix} \cdot (2-\lambda)$$

We compute the 2×2 determinant:

$$\det \begin{pmatrix} 3-\lambda & 2 \\ 2 & 3-\lambda \end{pmatrix} = (3-\lambda)^2 - 4$$

Characteristic equation:

$$(2-\lambda)((3-\lambda)^2 - 4) = 0$$

So either:

Case A: $2 - \lambda = 0 \Rightarrow \lambda = 2$.

Case B: $(3 - \lambda)^2 - 4 = 0$

$$\Rightarrow (3 - \lambda)^2 = 4$$

$$\Rightarrow 3 - \lambda = \pm 2.$$

- If $3 - \lambda = 2$, then $-\lambda = -1 \Rightarrow \lambda = 1$.
- If $3 - \lambda = -2$, then $-\lambda = -5 \Rightarrow \lambda = 5$.

The **eigenvalues** are:

$$\lambda_1 = 5, \lambda_2 = 2, \lambda_3 = 1$$

2. Principal Components

We already found the eigenvalues: 5, 2, 1. The largest eigenvalue is $\lambda_1 = 5$.

Solve:

$$\Sigma = \begin{pmatrix} 3 & 2 & 0 \\ 2 & 3 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

$$\Sigma - 5I = \begin{pmatrix} 3-5 & 2 & 0 \\ 2 & 3-5 & 0 \\ 0 & 0 & 2-5 \end{pmatrix} = \begin{pmatrix} -2 & 2 & 0 \\ 2 & -2 & 0 \\ 0 & 0 & -3 \end{pmatrix}$$

Let $v = \begin{pmatrix} x \\ y \\ z \end{pmatrix}$. Then $(\Sigma - 5I)v = 0$ gives:

$$\text{From row 1: } -2x + 2y + 0z = 0 \Rightarrow -2x + 2y = 0 \Rightarrow y = x.$$

$$\text{From row 2: } 2x - 2y + 0z = 0 \Rightarrow 2x - 2y = 0 \Rightarrow y = x \text{ (same condition).}$$

$$\text{From row 3: } 0x + 0y - 3z = 0 \Rightarrow -3z = 0 \Rightarrow z = 0.$$

So any eigenvector has the form: $v = \begin{pmatrix} t \\ t \\ 0 \end{pmatrix}$ with $t \neq 0$.

Choose $t = 1$ to get a simple direction: $v = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$.

$$\text{Its Euclidean norm is: } \|v\| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}.$$

$$\text{Normalized eigenvector: } v_1 = \frac{1}{\|v\|}v = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}.$$

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}$$

This v_1 is the direction of the **First Principal Component (PC1)**.

3. Explained Variance

Total variance is:

$$\sum_{i=1}^3 \lambda_i = 5 + 2 + 1 = 8$$

Variance explained by the first two principal components:

$$\frac{\lambda_1 + \lambda_2}{\sum \lambda_i} = \frac{5 + 2}{8} = \frac{7}{8} = 0.875$$

$$\boxed{\text{Cumulative explained variance (PC1 + PC2) = 87.5\%}}$$